

The Checklist

**RITAH KOMUHANGI
2026**

2

ABOUT

The Checklist is formerly an "ICDD Final Project" that was designed to help Ryder Ace stay motivated and develop consistent daily habits through a structured routine.

The checklist includes essential daily activities such as greeting after waking up, brushing teeth, taking a bath, and other similar tasks.

After completing each task, Ryder can press a button located next to the corresponding icon to mark it as done.

Each completed task earns him points, reinforcing positive behavior through a simple reward system.



3

ORIGINAL BUILD

i. HTML

The Checklist was structured in HTML using containers to place the columns, activities, buttons and so on.

ii. PYTHON

A points() function was defined in python to calculate the number points collected weekly using lists & dictionaries to store data, loops, variables & so on to assist data collection.

iii. CSS

Cascading Style Sheets handled the display and styling

iv. JAVASCRIPT

Javascript was responsible for implementation of the responsivity of the program.



4

Integrations

Object Oriented Programming

Initiated the 'Day' Class with attributes such as timeofday, activity and points as well as static and non-static methods.

```
finalproject > main.py > ...
1 import csv # Imports the csv module to read and write csv files
2 from pathlib import Path # Imports Path from pathlib to handle file paths
3 from datetime import datetime # Imports datetime so we can get the current date and time
4 import json # Imports json module to write data for the frontend
5 from tinydb import TinyDB # Imports tinydb to manage Database operations
6
7 # Main 'Day' Class to represent and manage daily activities
8 class Day:
9
10     def __init__(self, timeofday, activity, points=1):
11         self.timeofday = timeofday # Stores time of day
12         self.activity = activity # Stores activity name
13         self.points = points # Stores points earned for completing activity
14         self.is_completed = False # Tracks if activity is done
15
16     # Static method to determine current part of the day
17     @staticmethod
18     def get_timeofday():
19         hour = datetime.now().hour # Gets the current hour from the system clock
20
21         if 5 <= hour < 12:
22             return "Morning"
23         elif 12 <= hour < 18:
24             return "Afternoon"
25         else:
26             return "Evening"
27
28     # Method to convert Day object to a dictionary for JSON Export
29     def conv_to_dict(self):
30         return{
31             "timeofday": self.timeofday,
32             "activity": self.activity,
33             "points": self.points,
34             "is_completed": self.is_completed
35         }
36
37     # Method to display objects to user
38     def __repr__(self):
39
40         if self.is_completed:
41             status = "Done"
42         else:
43             status = "Pending"
44         return f"{self.activity} : {status}"
```

Integrations

File Operations

Accessing , Opening and Reading from the 'activities.csv' file.

Preparing data for JSON Export then writing it to the 'schedule_data.js' file.

```

105     # A dictionary to prepare data for JSON Export
106     exp_data = {
107         "current_timeofday": current_timeofday, "generated_at": datetime.now(),
108         "activities":{"Morning": [], "Afternoon": [], "Evening": []}}
109
110     # Adds all activities to export
111     for task in tasks:
112         exp_data["activities"][task.timeofday].append(task.conv_to_dict())
113
114     # Writes JSON File for frontend
115     json_path = Path(__file__).parent/"schedule_data.js"
116
117     completed_list = []
118     for task in tasks:
119         if task.is_completed == True:
120             completed_list.append(task.activity)
121
122     jsstring = json.dumps(completed_list) # Converts the list into JavaScript text string
123
124     # Writes list of completed activities to JSON file
125     with json_path.open(mode="w") as json_file:
126         json_file.write(f"var completedTasks = {jsstring};")
127

```

```

finalproject > main.py > ...
46     def main():
47         db = TinyDB("points_db.json") # Creates the TinyDB database file
48         tasks = [] # Empty list to store Day objects
49         file_name = "activities.csv" # Stores the CSV file name in a variable
50         path = Path.cwd() / file_name # Creates a full path to the file using the current working directory
51
52         with path.open(mode="r") as file: # Opens the CSV file in read mode
53             reader = csv.DictReader(file) # Creates a DictReader object treating each CSV Row as a dictionary
54
55             # Loops through each row, creates a Day Object then adds it to the tasks list
56             for row in reader:
57                 tasks.append(Day(timeofday="Morning", activity=row["Morning"].strip(), points=1))
58                 tasks.append(Day(timeofday="Afternoon", activity=row["Afternoon"].strip(), points=1))
59                 tasks.append(Day(timeofday="Evening", activity=row["Evening"].strip(), points=1))
60

```

6 Other Integrations



Virtual Enviroments

To store external libraries like TinyDB.



Database Management

To store and keep track of daily points

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(checklist) PS C:\Users\ReDI User\Documents\GitHub\The-Checklist---Ryder-Ace\finalproject> python main.py

-Morning Schedule
-
Is 'Greeting' finished?: yes
Is 'Brushing Teeth' finished?: yes
Is 'Taking a Bath' finished?: yes
Is 'Dress Up' finished?: yes
Is 'Make The Bed' finished?: no
Is 'Breakfast' finished?: no
Is 'Puzzle' finished?: no
Is 'Snack Time' finished?: no
Is 'Take A Walk' finished?: no

////////////////////////////////////
Greeting : Done
Brushing Teeth : Done
Taking a Bath : Done
Dress Up : Done
Make The Bed : Pending
Breakfast : Pending
Puzzle : Pending
Snack Time : Pending
```

CHECKLIST NOW

The Python backend now powers the checklist system by managing activities, tracking progress, and storing user achievements by performing the following tasks:

1. **Reads activity data from a CSV file**, so tasks can be easily added or changed without editing the code.
2. **Uses a Day class** to store each task as an object, helping to keep the code organized using OOP.
3. **Detects the current time of day** (Morning, Afternoon, Evening) so the right set of activities can be shown at the right time.
4. **Stores points in a TinyDB database**, so progress is saved even after closing the program.
5. **Records the date and time** each checklist session is completed, so progress can be tracked over time.
6. **Exports completed tasks to a JavaScript file**, which connects the backend to the website frontend.
7. **Links the backend to the website**, so the web page always shows updated tasks and progress.

CHALLENGES & SOLUTIONS

1. Converting Python Data to JSON for Frontend Display:

Review of the File Operations Notes and using the **W3Schools** Python JSON tutorial, helped me understand more clearly how to convert and re-convert Python data into JSON format using `json.dumps()` and `json.loads()`.

2. Fixing Instance and Static Method Issues:

After reviewing our class notes, I cleared up my confusion regarding instance versus static methods.

I was able to successfully implement a static method that runs independently without needing the `self` parameter, making the class design much cleaner.

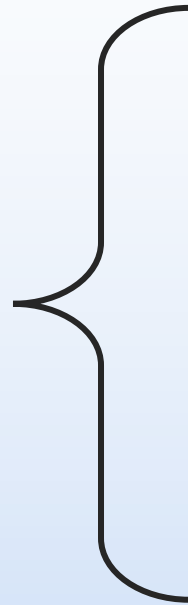
3. Displaying JSON Data on The Interface:

Prompted Gemini to help me understand step by step how i could source data from the 'schedule_data.js' and display it on the frontend.

IMPROVEMENTS

- I plan to add error handling and exception management to make the program more stable and prevent crashes when unexpected input or file issues occur.
- I also plan to move from TinyDB to a bigger online database like MySQL, so the system can handle more data and work better as the project grows.

Thank you



Ritah Komuhangi